



TP6 CDA PRF · RNCP 37873 — CONCEPTEUR
DÉVELOPPEUR D'APPLICATIONS (TP6 CDA)

COURS HTML5, CSS3 & BOOTSTRAP

HTML/CSS/Bootstrap

J1 — HTML essentiel + bases CSS

PROMO 2026-D03, CDA JS FULLSTACK IA

Formateur : Robin Hotton, rhotton@diginamic.fr

Formateur Robin HOTTON

Promotion 2026-D03

Version 1.2 — 2026

Agenda du jour

Matin — HTML essentiel

- I** — Histoire du Web
- II** — Bases HTML5 sémantique
- III** — Attributs
- IV** — Formulaires

Après-midi — Bases CSS

- VI** — Introduction CSS
- VII** — Visualiser la structure
- VIII** — Syntaxe & propriétés
-  **Bloc DigiShop initial**

À la fin de la journée, vous saurez :

-  Créer une page HTML5 sémantique
-  Styler un formulaire accessible
-  Déboguer dans DevTools
-  Thémer un site en variables CSS

I — Histoire du Web

Comprendre d'où vient HTML et pourquoi le W3C existe

D'après vous...

Pourquoi le Web est-il devenu un standard universel et **gratuit** alors qu'il aurait pu être un produit propriétaire ?

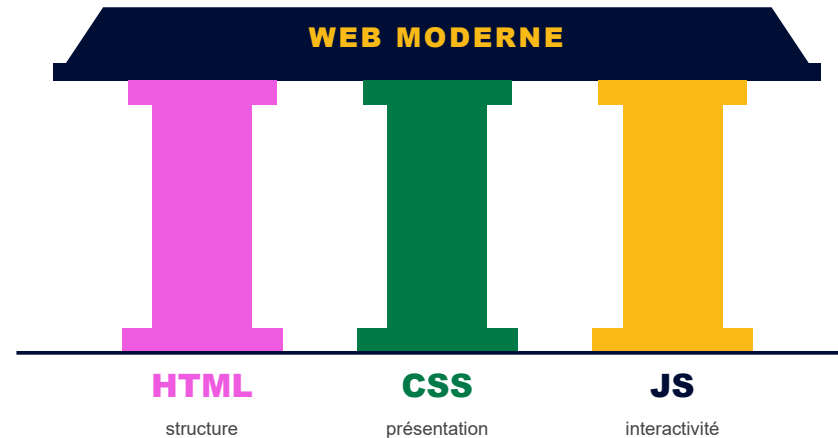
◆ UNE DÉCISION VISIONNAIRE

1993 — CERN libère le Web en **open source**

Avant — messageries propriétaires (MSN, AOL, CompuServe)

Après — **un seul Web** universel

Les 3 piliers du Web moderne



◆ MÉTAPHORE DE LA MAISON

HTML → murs et pièces (*titres, paragraphes, liens*)

CSS → peinture et mobilier (*présentation, mise en page*)

JS → électricité et plomberie (*clics, APIs, animations*)

Pratique — Inspect 30 sec

 **Sondage** — levez la main si vous avez déjà ouvert **F12** sur un site

EXERCICE ●, EN SÉANCE, 2 MIN

Ouvrir n'importe quel site dans Chrome → **F12** → onglet **Éléments**.

Identifiez :

Une balise `<header>` , `<nav>` ou `<main>`

Une feuille CSS chargée (onglet Réseau → filtre CSS)

Un script JS (onglet Réseau → filtre JS)

II — Bases HTML5 sémantique

Squelette d'une page et choix des balises

D'après vous...

Pourquoi recommande-t-on `<section>`, `<article>`, `<nav>` plutôt que des `<div>` partout ?

❌ Mauvaise pratique — soupe de div

```
<div class="header">
  <div class="nav">...</div>
</div>
<div class="main">
  <div class="article">
    <div class="title">Mon titre</div>
    <div class="content">...</div>
  </div>
</div>
<div class="footer">...</div>
```

◆ TROIS BONNES RAISONS D'UTILISER LE SÉMANTIQUE

Accessibilité — NVDA annonce *navigation* pour `<nav>`

SEO — Googlebot privilégie `<article>` pour les contenus autonomes

Maintenabilité — relire la structure sans CSS = lecture immédiate du sens

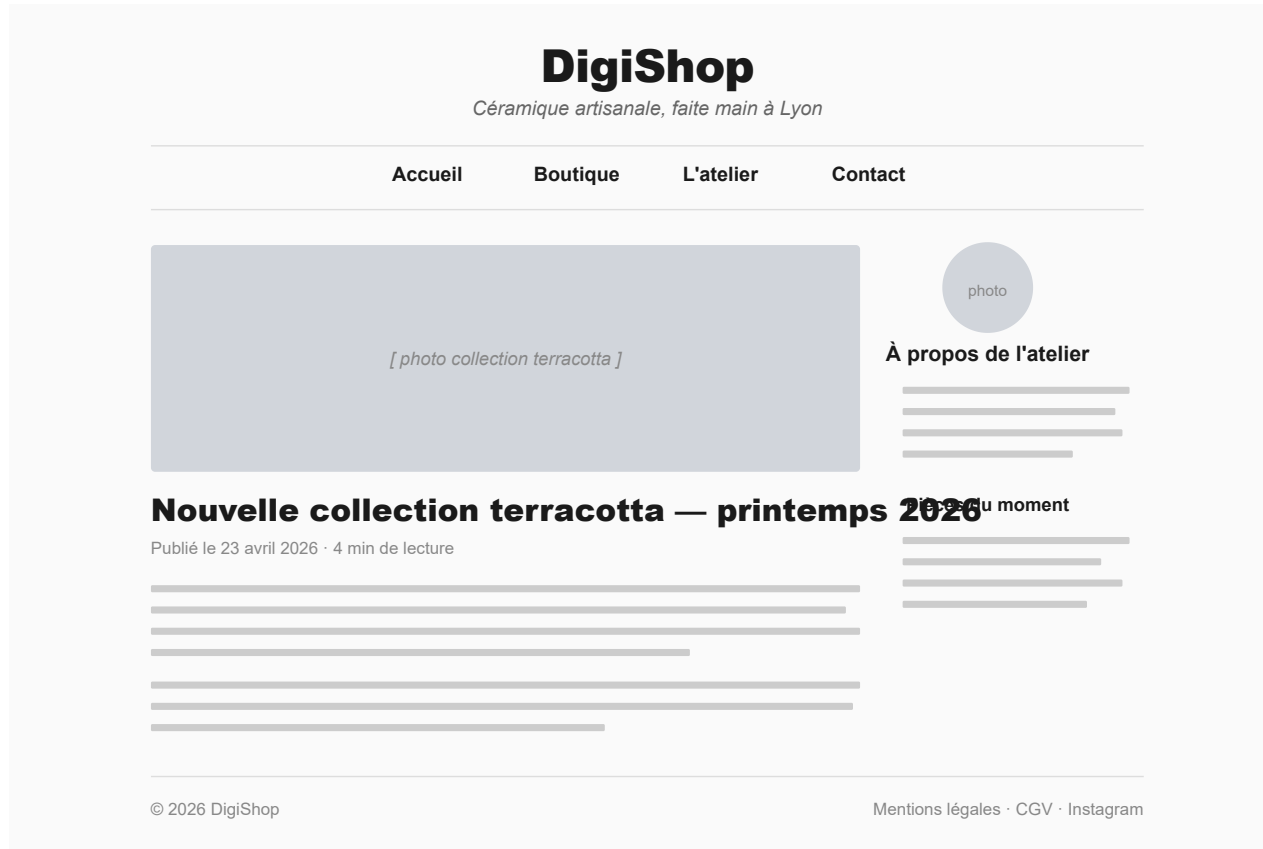
Balises sémantiques structurantes



Pratique — Squelette HTML5 minimal

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ma page</title>
</head>
<body>
  <header><h1>Titre</h1></header>
  <main>Contenu principal</main>
  <footer>Pied de page</footer>
</body>
</html>
```

Exercice — Reproduire la page



Correction — Découpage sémantique



Recommandations

◆ SÉMANTIQUE D'ABORD

Toujours préférer `<header>` , `<nav>` , `<main>` , `<article>` , `<aside>` , `<footer>` à un `<div>` quand le sens existe.

◆ UN SEUL `<MAIN>` PAR PAGE

C'est la zone du contenu **principal** unique. Erreur fréquente : en mettre plusieurs.

❗ HIÉRARCHIE DE TITRES

1 seul `<h1>` , puis `<h2>` , `<h3>` ... sans saut. Garantit l'accessibilité (lecteurs d'écran) et le SEO.

⚠ TOUJOURS VALIDER W3C

validator.w3.org en deux secondes — 0 erreur attendue avant de passer au CSS.

III — **Attributs HTML**

Identifier, styler, sécuriser

Un attribut, c'est quoi ?

Une **information supplémentaire** ajoutée à une balise, sous forme `nom="valeur"`.

```
<a href="https://mdn.dev" target="_blank">MDN</a>  
  
<input type="email" required>
```

◆ 3 RÔLES PRINCIPAUX

Identifier — `id`, `class` (référencer en CSS / JS)

Connecter — `href`, `src` (lier des ressources)

Sécuriser & optimiser — `alt`, `rel`, `loading`, `type` ...

D'après vous...

Q1 — `target="_blank"` sur un lien externe : pourquoi c'est un **risque de sécurité** ?

⚠ TABNABBING

`window.opener` exploitable → toujours `rel="noopener noreferrer"`.

Q2 — Image **décorative** : `alt=""` vide ou pas d'`alt` du tout ?

📄 ALT="" EXPLICITE

`alt=""` → lecteur d'écran **ignore**. Sans `alt` → URL lue à voix haute.

D'après vous...

Q3 — `loading="lazy"` sur **toutes** les images : bonne idée ou pas ?

 PAS SUR LE HERO

Lazy sur l'image **above the fold** → sanction LCP → mauvais SEO. Garder `loading="eager"` (défaut) sur le hero.

Attributs essentiels — par rôle

★ ID IDENTIFIER

`id` — **unique**

`class` — **réutilisable**

Hook CSS, JS, ancrés `#id`.

ℹ️ 🔗 CONNECTER

`href` — `<a>`, `<link>`

`src` — `<img>`, `<script>`

`action` — `<form>`

⚠️ 🔒 SÉCURISER & OPTIMISER

`alt` — a11y + SEO

`rel="noopener..."` — anti-tabnabbing

`loading="lazy"` — perf (hors hero)

`type` — input typé

Le piège n°1 : `id` vs `class`

◆ ID (UNIQUE)

```
<h1 id="accueil">Bienvenue</h1>
<a href="#accueil">Haut</a>
```

◆ CLASS (RÉUTILISABLE)

```
<p class="lead">Intro</p>
<p class="lead">Suite intro</p>
```

En CSS : `#accueil` cible l'`id`, `.lead` cible la `class`.

Pratique — Lien externe sécurisé

 **Démo DevTools** — onglet Lighthouse → audit Performance → identifier le **LCP**

```
<a href="https://mdn.dev"
  target="_blank"
  rel="noopener noreferrer">
  MDN Web Docs ↗
</a>




```

Exercice — Enrichir la page DigiShop

EXERCICE ● · `EX/03\_ATTRIBUTS.MD` · 15 MIN

Reprendre la page produite à l'exercice précédent et **enrichir** la boutique avec navigation cliquable, visuels optimisés et lien sortant sécurisé :

Navigation — liens `<a href>` cliquables sur les 4 entrées du menu

Image hero dans l' `<article>` — `alt` descriptif + `loading="eager"`

Image aside — `alt` descriptif + `loading="lazy"`

Lien externe Instagram dans le `<footer>` — `target="_blank"` + `rel="noopener noreferrer"`

Livrable : la page de l'EX/02 enrichie, Lighthouse encore au vert (LCP et a11y).

IV — Formulaires HTML

La porte d'entrée des données utilisateur

D'après vous...

Q1 — Qu'est-ce qu'un formulaire en HTML ?

DÉFINITION

Zone qui **collecte** des données utilisateur et les **envoie** au serveur (ou à JS). Délimitée par `<form>`.

Q2 — De quoi est-il composé ?

BRIQUES PRINCIPALES

`<form>` — conteneur

`<label>` — texte associé à un champ

`<input>`, `<textarea>`, `<select>` — champs

`<button type="submit">` — envoi

Q3 — Le `placeholder` peut-il **remplacer** le `<label>` ?

Anatomie d'un formulaire

Se connecter

à votre espace DigiShop

Email

Mot de passe

☐ Se souvenir de moi [Mot de passe oublié ?](#)

Se connecter

Pas encore de compte ? [Créer un compte](#)

Classiques

Texte — `text` , `password`

Choix — `checkbox` , `radio`

Action — `submit`

HTML5 utiles

Texte typé — `email` , `tel`

Date / heure — `date` , `time`

Nombre — `number` + `min` / `max`

Validation — `required` , `pattern`

Code — formulaire de connexion accessible

```
<form method="post" action="/login">

  <label for="loginEmail">Email</label>
  <input id="loginEmail" name="login" type="email" required>

  <label for="loginPwd">Mot de passe</label>
  <input id="loginPwd" name="pwd" type="password" required>

  <label>
    <input id="rememberMe" name="remember" type="checkbox">
    Se souvenir de moi
  </label>

  <button type="submit">Se connecter</button>

</form>
```

Attribut	Valeur	Rôle
id	loginEmail	Identifiant DOM — relie <code><label for></code>
name	login	Clé envoyée au backend HTTP
type	email	Validation + clavier navigateur

Exercice — Formulaire d'inscription DigiShop

Créer un compte

Rejoignez DigiShop

Prénom

Email

Mot de passe

8 caractères minimum

Date de naissance

☐ J'accepte les **CGU**

S'inscrire

Déjà inscrit ? [Se connecter](#)

EXERCICE •

`EX/04\_FORMULAIRES.MD` • 20 MIN

Reproduire la maquette en HTML5 + valider sans une ligne de JS :

Inputs typés `email` , `password` , `date`

Attributs `required` , `minlength` , `pattern`

Binding `label/for/id` propre

● **Défi** — confirmation mot de passe + comparaison JS.

Test : soumettre vide → erreurs natives s'affichent seules.

VI — Introduction CSS

Box-model et intégration au HTML

✦ CHAP V SAUTÉ — CHEATSHEET EN AUTONOMIE

Référentiel des balises HTML5 par famille (structure, texte, médias, tableaux, formulaires). À consulter au fil des TP, **pas à apprendre par cœur**.

CSS, c'est quoi ?

Le langage qui **présente** ce que HTML structure. Trois questions pour démarrer.

Q1 — Où écrit-on les règles CSS ?

❗ 3 MÉTHODES — UNE SEULE À RETENIR

Externe — `<link rel="stylesheet" href="styles.css">`  recommandé

Interne — `<style>...</style>` dans `<head>` (cas isolés)

Inline — `style="..."` sur la balise (à éviter)

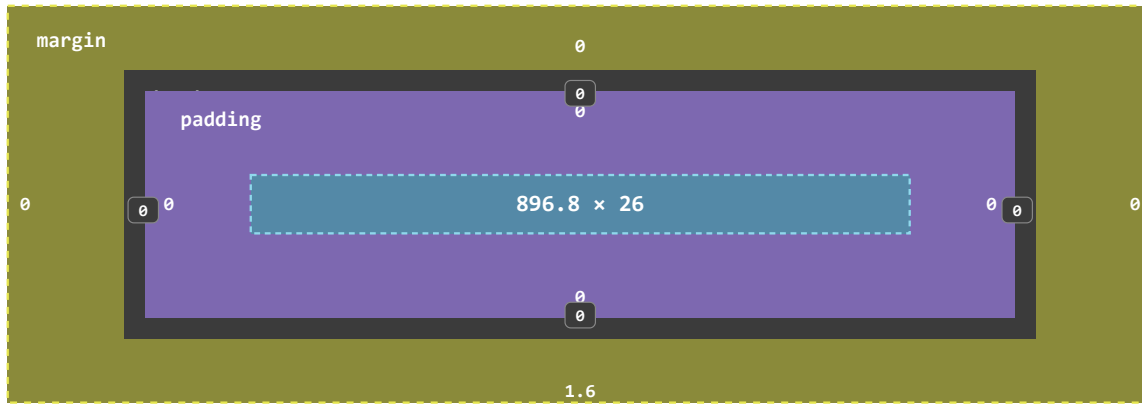
Q2 — Comment cibler un élément ?

◆ UNE RÈGLE = SÉLECTEUR + BLOC

```
h1 { color: navy; } /* par balise */  
.lead { font-size: 1.2rem; } /* par class */  
#header { background: #fff; } /* par id */
```

Q3 — Pourquoi `* { box-sizing: border-box }` est devenu un réflexe pro ?

Le box-model — anatomie et reset



4 zones imbriquées

content — texte, image

padding — marge **intérieure**

border — la bordure

margin — marge **extérieure**

```
/* Reset universel - à coller en tête de tout style.css */
*, *::before, *::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  line-height: 1.6;
  color: #1f2937;
}
```

Exercice — Box-sizing en action

EXERCICE `EX/06\_BOX-SIZING.MD` • ● 5 MIN + ● 10 MIN

Sentir **dans son code** la différence entre `content-box` (modèle historique) et `border-box` (modèle utilisé partout en pratique).

- Recopier le gabarit `.boite` + variantes `.type-a` / `.type-b` et observer les 2 rendus côte à côte.
- Prédire à la main la largeur réelle de 4 boîtes (cas A → D), puis vérifier dans DevTools.

Vérif : la prédiction colle au pixel près sur les 4 cas — sinon retour au schéma.

VII — Visualiser la structure

Debug visuel et DevTools

D'après vous...

Q1 — Comment **voir instantanément** la structure d'une page web sans en connaître le HTML ?

◆ LE SNIPPET MAGIQUE

```
* { outline: 1px solid red; }
```

À coller via la **console DevTools** — tous les blocs apparaissent.

Q2 — Pourquoi `outline` et pas `border` ?

◆ OUTLINE NE CASSE PAS LE LAYOUT

`border` modifie la **taille** → décale le voisinage

`outline` est dessiné **à l'extérieur**, n'occupe aucun espace

Q3 — Quel onglet DevTools révèle les **valeurs CSS finales** ?

❗ COMPUTED

Panneau **Computed** (à côté de Styles) → valeurs résolues après cascade et héritage.

DevTools — les 4 onglets à connaître

Éléments + Styles

Inspecter et modifier le HTML / CSS **en live**.

Panneau **Computed** → valeurs CSS finales (cascade résolue).

Schéma **box-model** interactif en bas.

Console

`console.log(...)` , erreurs JS, exécuter du JS sur la page.

Injecter un snippet CSS one-shot.

Réseau

Voir chaque ressource : `.html` , `.css` , `.js` , images.

Statut, taille, temps de chargement, ordre.

Lighthouse

Audit automatique : **performance**, **accessibilité**, **SEO**, **bonnes pratiques**.

Rapport notable en 30 secondes.

Exercice — Inspecter une vraie page

EXERCICE `EX/07\_DEVTOOLS-INSPECT.MD` • ● 5 MIN + ● 15 MIN

Apprivoiser **DevTools** comme un outil de debug et de mesure — pas juste un inspecteur curieux.

● Sur une page HTML simple : ouvrir DevTools, lire la `font-size` calculée d'un `<h1>` dans **Computed**, puis injecter `* { outline: 1px solid red }` via la Console.

● Modifier `padding` + `background` à la volée, repérer le `color` hérité, lancer un audit **Lighthouse**, observer la cascade **Network** sur reload.

Aucune modif persistante : tout vit dans le navigateur — refresh = retour à zéro.

VIII — Syntaxe & propriétés CSS

5 familles à maîtriser pour démarrer

D'après vous...

Q1 — Quelle unité pour la taille de police : `px` ou `rem` ?

◆ REM — ACCESSIBILITÉ AVANT TOUT

L'utilisateur peut **agrandir la taille par défaut** du navigateur (zoom typo)

`rem` s'adapte proportionnellement · `px` figé → **barrière a11y**

Q2 — Combien de **familles** de propriétés couvrent 80 % du CSS quotidien ?

① CINQ

Couleurs — `color` , `background`

Dimensions — `width` , `height`

Espacements — `margin` , `padding` , `gap`

Typographie — `font-*` , `line-height`

Positionnement — `display` , `position` , Flex/Grid

Le piège classique — centrer horizontalement

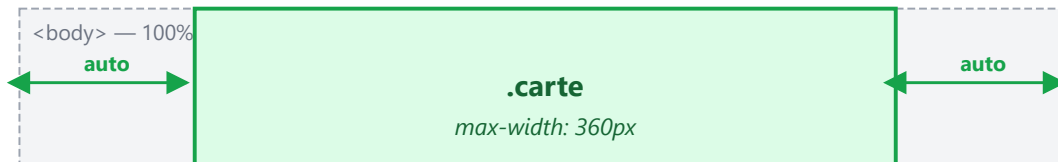
✗ Sans largeur

```
.carte { margin: 0 auto; }
```



✓ Avec max-width

```
.carte { max-width: 360px; margin: 0 auto; }
```



MARGIN AUTO A BESOIN D'UNE LARGEUR

Sans `width` ou `max-width`, un bloc occupe déjà **100 %** de la place : `auto` n'a rien à répartir.



INLINE ? TEXT-ALIGN

`margin: auto` ne centre **que les blocs**.

Pour du texte ou des `<span>` : `text-align: center` **sur le parent**.

Anatomie d'une règle CSS

```
/* sélecteur { propriété: valeur; ... } */  
h1.titre-marque {  
  color: #c0186e;      /* propriété : valeur */  
  font-size: 1.5rem;  
  margin-bottom: 1rem;  
}
```

◆ SÉLECTEUR

Qui cibler : balise, `.class`, `#id`,
combinaison.

❗ PROPRIÉTÉ

Quoi modifier : la centaine de propriétés
CSS du standard.

◆ VALEUR

Avec quoi : `1rem`, `#c0186e`, `flex`,
`system-ui` ...

Code — variables CSS + carte produit

```
:root {
  --rose: #c0186e;
  --texte: #1f2937;
  --space: 1rem;
}

body {
  font-family: system-ui, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: var(--texte);
  margin: 0;
}

.carte {
  background: white;
  max-width: 320px;
  margin: 0 auto;          /* centré horizontalement */
  padding: var(--space);
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.08);
}
```

Exercice — Carte produit DigiShop

EXERCICE `EX/08\_SYNTAXE-COULEURS.MD` • ● 5 MIN + ● 10 MIN

Construire une **carte produit DigiShop** en s'imposant **zéro couleur ni police hardcodée** — uniquement des `var(--x)` définies dans `:root`.

- Poser la **palette en variables CSS** (`:root`) + appliquer fond et police au `body`.
- Écrire `.carte`, `.carte-titre`, `.carte-prix`, `.carte-btn` — couleurs et police **uniquement** via `var(--x)`.
- **Défi** — échelle typo cohérente (`--fs-xs` à `--fs-3xl`, plus aucune `font-size` en dur).



● **Récap J1 — DigiShop initial**

Mini-TP autonomie guidée — réinvestir tous les concepts du jour

● Récap J1 — Mission

FORMATIF · 65 MIN EN SÉANCE · AUTONOMIE GUIDÉE — `EX/DIGISHOP/01\_INITIAL.MD`

Construire la page d'accueil **DigiShop** (atelier céramique à Lyon) en 5 étapes :

1. Structure HTML5 sémantique (header / main / 3 articles produit / aside / footer)
2. Images avec `alt` descriptif + `loading` eager/lazy
3. Reset CSS + `box-sizing: border-box` global
4. Palette de marque en **variables CSS** (terre cuite, crème, vert sauge)
5. Validation W3C + Lighthouse Accessibility ≥ 90

◆ DÉFIS ●

Pour les rapides — étoffer DigiShop à la maison :

Page boutique (catalogue produits)

Formulaire contact complet

Mode sombre via `prefers-color-scheme`

Conclusion J1

À retenir + cap sur J2

À retenir — J1

HTML

Sémantique > soupe de `<div>`

Un seul `<h1>`, un seul `<main>`

Attributs essentiels — `alt`, `lang`, `rel`, `loading`

Formulaires : un `<label>` par champ, jamais `placeholder` seul

CSS

Reset `* { box-sizing: border-box }`

Tailles en `rem`, pas en `px`

4 familles — couleurs · dimensions · espacements · typo

DevTools (Computed + outline rouge) en réflexe

◆ LA MÉTAPHORE DE LA MAISON TIENT

HTML → murs · **CSS** → déco · **JS** → électricité (*la semaine prochaine*)

Demain — J2 : CSS avancé

Matin

Sélecteurs avancés — combiner, cibler avec précision

Cascade & spécificité — qui gagne quand deux règles s'opposent

Flexbox + Flexbox Froggy 🐸

Après-midi

Grid + Grid Garden 🌱

Responsive mobile-first — adapter aux écrans

● **Bloc DigiShop responsive** (95 min)

◆ CE SOIR, À LA MAISON (OPTIONNEL)

Défi ● J1 : F12 → onglet **Network** sur un site préféré, observer méthodes/statuts/types de ressources

Étoffer son DigiShop (sections ● non terminées)

Installer l'extension **Live Server** pour VS Code



Cofinancé par
l'Union européenne



TP6 CDA PRF · RNCP 37873 — CONCEPTEUR
DÉVELOPPEUR D'APPLICATIONS (TP6 CDA)

COURS HTML5, CSS3 & BOOTSTRAP

Merci !

À DEMAIN POUR J2 — CSS AVANCÉ · BOOTSTRAP

Questions, retours, demandes de rattrapage — c'est le moment.

Robin Hotton · rhotton@diginamic.fr

Formateur Robin HOTTON

Promotion 2026-D03

Version 1.2 — 2026